**Collaboration, Quality, Test**
WS2025/2026
Prof. Dr. Jóakim von Kistowski

# Exercise 5.1 – Mocking Components in automated Tests

Learning objectives: Understand how to create and use a mock in Unit Tests

Today, we add a second component (class) to our “triangle” software. This component uses our original triangle class and must take care of stubbing/mocking it in its unit test.

The structure of this exercise is three-fold:

1. Refactor the Triangle class to allow for mocking
2. Implement the new component/class
3. Implement some unit tests using mocks

In this exercise, it is recommended that everyone code on their own device. Feel free to discuss and learn from the experiences of your neighbors.

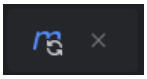
## Part 1 – Setting up and Refactoring

1. Open the pom.xml file and add the following dependencies to the list of dependencies:

```
<dependencies>
  ….
  [EXISTING DEPENDENCIES]
  ….

  <dependency>
      <groupId>org.mockito</groupId>
      <artifactId>mockito-core</artifactId>
      <version>5.21.0</version>
      <scope>test</scope>
  </dependency>
  <dependency>
      <groupId>org.mockito</groupId>
      <artifactId>mockito-junit-jupiter</artifactId>
      <version>5.21.0</version>
      <scope>test</scope>
  </dependency>
</dependencies>
```

Note: You may have to downgrade JUnit from version 6 to version 5.14.1.

2. Click on the refresh button that appears to the top right or use the keyboard shortcut ctrl+shift+O 
3. Change the `numberOfEqualSides`-method in your Triangle class to not be static.
4. Modify the Triangle Tests so that they run and pass
5. Discuss with your neighbors: Why was it a good idea to parameterize the Triangle tests?

## Part 2 – Implement the New Class

Next, we implement the class named `TriangleFromPoints`. The class has a method that takes six two-dimensional points, representing the three points of a triangle, calculates the sides' length (rounding to the nearest integer) and then calls our original *numberOfEqualSides* method in the *Triangle* class.

1. In your Java package named *de.thab.cqt.triangle* under the folder *src/main/java* create a class named: *TriangleFromPoints*
2. Implement a way to pass an instance of the *Triangle* class to *TriangleFromPoints*. Discuss with your neighbors: What ways did you find. What do you think is the best way to pass the instance?
3. Create the following method in your class TriangleFromPoints:
   ```
   public int numberOfEqualSides(int x0, int y0, int x1, int y1, int x2, int y2)
   ```
   In this method:
   1. Calculate the Euclidean distance between each of the points using the formula
      $$d(p, q) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2}.$$
   2. Round the result to the nearest integer (use *`Math.round()`*)
   3. Pass the resulting sides to *Triangle*'s *numberOfEqualsSides* method
   4. Return the result

## Part 3 – Writing Unit Tests

We are going to test the *TriangleFromPoints* class in isolation. We are using the Java library *Mockito*.

1. In the newly created package, create a Java class with the name *TriangleFromPointsTest*
2. To enable Mockito in this test add the following Annotation on your class, so that your code looks like this:
   ```
   @ExtendWith(MockitoExtension.class)
   class TriangleFromPointsTest {
   ```
3. In your test class create a mock for the Triangle class. The easiest way to do so is to add a class attribute (class level variable) with the type of Triangle and add the

`@Mock`-Annotation to it. This will tell our test runner to set up a mock for this variable. Your class should now look something like this:

```
@ExtendWith(MockitoExtension.class)
class TriangleFromPointsTest {
    @Mock
    private Triangle triangle;
    ...
```

4. Before you start writing tests, discuss with your neighbors:
   - What are the test cases you would test in this class?
     Consider: You do not want to test the behavior of the *Triangle* class!
5. Write a unit test. Pass the mock *Triangle* instance into the *TriangleFromPoints* instance to be tested. To specify the Mock's behavior in the test use the following method:

```
Mockito.when(<MOCK>.<methodToBeMocked>(<expectedParameters>))
 .thenReturn(mockResult);
```

6. Write additional tests for your other test cases.

# Exercise 5.2 – Test Levels

Learning objectives: Foster a deeper understanding of the different test levels

## Task 1 – Test Level Definition

Look up the definition for the following terms in the ISTQB-Glossary:

- Component test

  _____

  _____

- Integration test

  _____

  _____

- System test

  _____

  _____

- Acceptance test

  _____

  _____

## Task 2 – Test Level Comparison

Compare the test level's main features.

| Criterion | Component test | Integration test | System test | Acceptance test |
|---|---|---|---|---|
| Test objectives | | | | |
| Test basis | | | | |
| Typical test objects | | | | |
| Test tools | | | | |
| Test environment | | | | |
| Typical failures | | | | |
| Notes | | | | |